

HashiCorp Vault 1.21 Community Edition as a Self-Hosted Identity Provider: The Complete Production Authn/Authz Reference

TL;DR

- **Vault 1.21 CE is a complete self-hosted authn/authz backbone:** trust Azure AD/Entra ID via the OIDC *auth* method (broker-in), map Entra security groups to Vault **external groups** via group-aliases keyed on the Azure group **object GUID**, attach one least-privilege ACL policy per group, then run Vault's own **OIDC provider** (`identity/oidc`) to mint RS256 JWTs your FastAPI/React apps verify via JWKS — all free in Community Edition.
 - **Three 1.21-era changes you MUST handle:** (1) `disable_mlock` has **no default and is required** with Integrated Storage (Vault won't boot otherwise), and the official container image had `cap_ipc_lock` removed at build time, so set `disable_mlock = true` + disable/encrypt swap and drop the IPC_LOCK cap; (2) `sys/generate-root`, `sys/rekey`, and DR-token endpoints are now **authenticated by default**; (3) Vault now rejects/redirects **non-canonical paths** and a new `recover` ACL capability exists alongside create/read/update/patch/delete/list/sudo/deny.
 - **What's Enterprise-only (cannot use on CE):** Sentinel EGP/RGP & control groups, namespaces, performance/DR replication, **automated** Raft snapshots, step-up/login-enforcement MFA beyond basic TOTP, seal-wrapping, multi-seal HA, HSM support, entropy augmentation. **Free on CE:** ACL policies, all auth methods, the identity/OIDC provider, Integrated Storage (Raft) + manual snapshots, transit & cloud-KMS auto-unseal, audit devices, **both rate-limit and lease-count quotas** (`sys/quotas`), user-lockout, and basic TOTP login MFA.
-

Key Findings

1. **Architecture:** Every request flows HTTP listener → barrier (AES-256-GCM decrypt) → routing to a mount → token/identity resolution → **policy ACL evaluation (deny-by-default, deny-wins, most-specific-path)** → backend → audit log (request **blocks** if no audit device can write). The cryptographic barrier protects a keyring; the **root key** encrypts the keyring's encryption key and is itself protected by Shamir shares (manual unseal) or wrapped by a KMS/transit seal (auto-unseal).
2. **Auth model:** auth methods only *verify identity and assign token + identity policies*; the Identity system (entities, aliases, internal/external groups) sits across all auth mounts and is where you centralize authorization. Tokens are the auth "currency."
3. **For 3 human roles + machines:** humans via Azure OIDC → 3 external groups (users/staff/admins) → 3 templated least-privilege policies; machines via **AppRole** issuing

batch tokens with response-wrapped, CIDR-bound, use-limited SecretIDs.

4. **Vault-as-OIDC-provider** issues standard RS256 ID tokens with `.well-known/openid-configuration` + JWKS; FastAPI verifies signature + iss/aud/exp; React uses auth-code+PKCE.
 5. **Production substrate**: Integrated Storage (Raft) + transit or cloud-KMS auto-unseal + end-to-end TLS (no TLS termination at the LB) + **two** audit devices on independent storage + revoked root token.
-

Details

1. Core Concepts & Architecture (mechanism-deep)

The cryptographic barrier. All data written to storage passes through a barrier encrypting with **AES-256-GCM** (256-bit Galois/Counter Mode authenticated encryption). The storage backend sees only ciphertext. The barrier is unsealable only after the **root key** is reconstructed.

Keyring, root key, encryption key.

- The **encryption key** (a.k.a. barrier/data-encryption key) actually encrypts/decrypts data and lives in the **keyring**.
- The **root key** encrypts the keyring. (HashiCorp renamed "master key" → "root key"; older CLI output still says "master key.")
- **Unseal keys (Shamir)** reconstruct the root key. With auto-unseal, the root key is encrypted by the seal (KMS/transit) and **recovery keys** are generated instead.
- Key rotation: `sys/rotate` rotates the *encryption key* (new writes use the new key; old keys retained to read old data); `operator rekey` generates new *unseal/recovery* shares; root-key rotation happens via rekey.

Shamir Secret Sharing internals. At `operator init` you choose `-key-shares` (total, default 5) and `-key-threshold` (default 3). Vault splits the root key (or, under auto-unseal, the recovery key) into N shares over a finite field; any `threshold` shares reconstruct the secret, fewer reveal nothing. Distribute shares to different humans; odd thresholds avoid split decisions.

Seal/unseal state machine. Boot → **sealed** (no encryption key in memory; only `sys/health`, `sys/seal-status`, `sys/unseal`, etc. work). Provide threshold unseal keys (or KMS auto-unseals) → **unsealed** (active or standby). `operator seal` returns to sealed; a restart re-seals. Under auto-unseal the node unseals itself on restart with no human.

Storage backend abstraction. Vault persists encrypted blobs to a pluggable backend (file, Raft/Integrated Storage, Consul, etc.); the backend never sees plaintext. HA requires a backend that supports it; Integrated Storage (Raft) is the HashiCorp-recommended, officially supported HA backend.

Identity across auth methods. Each successful login creates/looks-up an **entity** (canonical identity) with an **alias** unique to `(mount_accessor, alias_name)`. Policies attach to the *entity* and to *groups*; these "identity policies" are evaluated **at request time** (dynamic), unlike token policies fixed at issue time.

Tokens as currency. Every authenticated request carries a token (`X-Vault-Token`); auth methods exchange external credentials for a token, and everything downstream is token-authorized.

Request lifecycle. HTTP request → listener (TLS) → token store resolves token → computes identity (entity+groups) → **ACL evaluation** merges token policies + identity policies → most-specific path match, deny-wins → route to mount/backend → response → audit broker writes to all devices (must succeed on ≥1 or the request is **blocked**).

Path-based API & verbs. Everything is a path under `/v1/`. HTTP verbs map to capabilities: GET→read, POST/PUT→create/update, DELETE→delete, PATCH→patch, **LIST** (or `GET ?list=true`)→list. Capabilities map to the *HTTP verb, not the semantic action* (e.g., generating DB creds is a `read`/GET).

Mounts & immutable mount accessor. Auth methods mount under `auth/<path>/`; secrets engines under `<path>/`. Each mount gets an immutable **accessor** (e.g., `auth_oidc_1a2b3c4d`) used in entity/group aliases and policy templating; the accessor survives renames.

sys/ backend. The reserved `sys/` mount exposes system endpoints: `sys/auth`, `sys/mounts`, `sys/policies/acl`, `sys/leases`, `sys/audit`, `sys/health`, `sys/seal-status`, `sys/rotate`, `sys/quotas`, `sys/storage/raft/*`, etc.

1.21 path canonicalization. Per the CHANGELOG: "core: Vault now rejects paths that are not canonical, such as paths containing double slashes (`path//to/resource`)" — later refined so Vault **redirects** non-canonicalized paths containing `/./`, `/../`, or `//` to a cleaned path. Also critical for policy authors: "**core/acl: LIST requests with a trailing slash now correctly respect more-specific deny policies. Previously, a deny on `path 'kv/*' { deny }` could be bypassed for `LIST kv/private/` if a broader allow `path 'kv/*'` also existed.**" Audit your LIST/deny policies after upgrade.

2. All Auth Methods

HUMAN auth

OIDC / JWT (`auth/oidc`, `auth/jwt`) — the centerpiece for your Azure AD humans.

Config endpoint `auth/oidc/config` parameters (defaults):

- `oidc_discovery_url` (string) — issuer base; for Entra `https://login.microsoftonline.com/<tenant_guid>/v2.0`

- `oidc_client_id` (string) — Entra Application (client) ID
- `oidc_client_secret` (string) — Entra client secret (confidential client; omit for public+PKCE)
- `oidc_discovery_ca_pem` (string: "") — CA for discovery TLS
- `jwtks_url`, `jwtks_ca_pem` (strings) — static JWKS alternative to discovery
- `oidc_response_mode` (string) — `query` or `form_post`
- `oidc_response_types` (string) — default `code`
- `bound_issuer` (string: "")
- `default_role` (string) — used when none specified at login
- `provider_config` (map) — for Azure `{"provider": "azure"}`; accepts `fetch_groups` (bool: false, **JWT login flow only**)
- `namespace_in_state` (bool: **true**)

Role endpoint `auth/oidc/role/<name>` parameters (defaults):

- `role_type` (string: "oidc") — `oidc` (browser) or `jwt` (direct JWT)
- `user_claim` (string: **required**) — claim → entity alias name (e.g., `sub`/`email`)
- `user_claim_json_pointer` (bool: false)
- `bound_audiences` (array) — required for `jwt` roles with an `aud`; since 1.17 must exactly match an `aud`
- `bound_subject` (string: "")
- `bound_claims` (map); `bound_claims_type` (string: "string" — or "glob")
- `groups_claim` (string) — Azure: `groups`
- `claim_mappings` (map) — copy claims → entity alias metadata
- `oidc_scopes` (array) — Azure `https://graph.microsoft.com/.default profile`
- `allowed_redirect_uris` (array: required for oidc)
- `clock_skew_leeway` (int/dur) — defaults to **60s** if 0; `-1` disables
- `expiration_leeway` / `not_before_leeway` (int/dur) — default **150s** if 0; `-1` disables (jwt roles)
- `verbose_oidc_logging` (bool: false) — logs raw token at debug; **never in prod**
- `max_age` (int)
- shared token params: `token_ttl` (0=system default), `token_max_ttl` (0), `token_policies` ([]), `token_bound_cidrs` ([]), `token_explicit_max_ttl` (0), `token_no_default_policy` (false), `token_num_uses` (0), `token_period` (0), `token_type` ("default")

OIDC login redirect flow & callback URLs.

- **UI callback:** `<VAULT_ADDR>/ui/vault/auth/oidc/oidc/callback`

- **CLI callback:** `http://localhost:8250/oidc/callback` — `vault login -method=oidc` spins up a local listener on **port 8250**, opens the browser to the IdP authorize URL, receives the code on 8250, posts code+state back to Vault, gets a token.
- Both URIs must be in the role's `allowed_redirect_uris` AND in the IdP's allowed callback list; for the UI, keep them lowercase to match the browser URL.

userpass (`auth/userpass`) — built-in username/password (bcrypt-hashed); good for break-glass. Subject to user-lockout. Params: `password`, `token_*`. Use only with strong policies + MFA.

LDAP (`auth/ldap`) — connect to AD/LDAP. Key params: `url`, `userdn`, `userattr`, `groupdn`, `groupattr`, `groupfilter`, `binddn`, `bindpass`, `starttls`, `insecure_tls`, `certificate`, `userfilter`, plus `token_*`. LDAP groups → Vault external groups via group-aliases (alias name = LDAP group name). Subject to user-lockout.

GitHub (`auth/github`) — org/team based. Params: `organization`, `base_url`, `token_*`. Teams → external groups (alias name = team slug).

MACHINE auth

AppRole (`auth/approle`) — recommended for your FastAPI/agentic services. Role endpoint `auth/approle/role/<name>` parameters (defaults):

- `bind_secret_id` (bool: **true**)
- `secret_id_bound_cidrs` (array: [])
- `secret_id_num_uses` (int: **0** = unlimited)
- `secret_id_ttl` (string: "" = no expiry)
- `local_secret_ids`/`enable_local_secret_ids` (bool: false; set only at creation)
- `token_ttl` (0), `token_max_ttl` (0), `token_policies` ([]), `token_bound_cidrs` ([]), `token_explicit_max_ttl` (0), `token_no_default_policy` (false), `token_num_uses` (0), `token_period` (0), `token_type` ("default" → service unless changed; set **batch** for machines)
- `alias_metadata` (map: nil)

Endpoints: `.../role/<n>/role-id` (read RoleID — stable, like a username); `.../role/<n>/secret-id` (generate SecretID — new each time; returns `secret_id` + `secret_id_accessor`); `.../role/<n>/custom-secret-id` (push mode); `.../role/<n>/secret-id/lookup|destroy` (via accessor). At login supply `role_id` + `secret_id`. SecretID-create accepts `ttl`, `num_uses`, `cidr_list`, `token_bound_cidrs` overrides (each ≤ the role's value).

- **Pull vs push:** pull = Vault generates SecretID (preferred); push = you supply via `custom-secret-id`.
- **Response wrapping for delivery:** generate the SecretID wrapped (`-wrap-ttl`), deliver the single-use wrapping token via a *separate channel* from the RoleID; consumer unwraps once.

Tamper-evident (a pre-unwrapped token fails).

- `secret_id_accessor`: manage/delete a SecretID without its value.

JWT auth — same plugin as OIDC, `role_type=jwt`: caller presents a pre-obtained JWT; Vault validates the signature against `jwtks_url`/`oidc_discovery_url`/static `jwt_validation_pubkeys`. No browser. Good for CI and federated workloads.

Kubernetes (`auth/kubernetes`) — config params (defaults): `kubernetes_host` (required), `kubernetes_ca_cert` (""), `token_reviewer_jwt` (""), `pem_keys` ([]), `issuer` (" — **deprecated**), `disable_iss_validation` (bool: **true** for new mounts since 1.9 — **deprecated**), `disable_local_ca_jwt` (bool: false), `alias_name_source` ("serviceaccount_uid"), `use_annotations_as_alias_metadata` (false). In-pod, `kubernetes_ca_cert` and `token_reviewer_jwt` default to the pod's local CA + SA token.

TLS cert auth (`auth/cert`) — mutual TLS; client cert (or CA) bound to a role, matching CN/SAN/OU. Good for service mesh / device identity.

AWS (`auth/aws`) — IAM (signed STS GetCallerIdentity) and EC2 (instance identity doc/PKCS7) modes; bind by account, role ARN, AMI, etc. **Azure** (`auth/azure`) — uses Azure MSI; validates `bound_subscription_ids`, `bound_resource_groups`, `bound_service_principal_ids`, and the `xms_mirid`/`xms_az_rid` claims of the MSI token. **GCP** (`auth/gcp`) — IAM and GCE-metadata modes.

The "secret zero" / secure introduction problem. Every machine auth ultimately needs *some* initial credential. Best practice: AppRole RoleID baked into image/config; SecretID delivered just-in-time, **response-wrapped**, with short `secret_id_ttl`, `secret_id_num_uses=1`, CIDR-bound; use **Vault Agent auto-auth** to manage the handshake and token lifecycle. Platform-native methods (K8s/AWS/Azure/GCP) eliminate a static secret entirely by trusting the platform's attestation.

3. Identity Secrets Engine (the core)

Entities (`identity/entity`): `name`, `id` (canonical_id), `metadata` (map), `policies` (array), `disabled` (bool). One entity = one human/machine across all aliases.

Entity aliases (`identity/entity-alias`): `name` (login identifier from `user_claim`), `canonical_id`, `mount_accessor`, `custom_metadata` (map, 1.9+). **Constraint: two aliases on the same mount cannot map to one entity** (a userpass alias and an LDAP alias can share an entity, but not two userpass-mount aliases). On first login Vault auto-creates the entity+alias. **Entity merging** combines duplicates.

Internal vs external groups (`identity/group`): `name`, `id`, `type` ("internal"|"external"), `metadata`, `policies`, `member_entity_ids`, `member_group_ids`. Internal: you set membership explicitly.

External: membership computed automatically from the IdP at login/renewal; exactly **one** alias;

cannot have `(member_entity_ids)`. Groups can nest `(member_group_ids)` for inheritance — keep nesting shallow.

Group aliases `(identity/group-alias)`: `(name)` (the external group's identifier — **for Azure the group OBJECT GUID**, for LDAP the group name, for GitHub the team slug), `(mount_accessor)`, `(canonical_id)`.

How external group membership is computed: at login/renewal the auth method emits the user's groups `(groups_claim)`; Vault matches each emitted value against group-aliases **on that mount accessor**; matches add the entity to those external groups; group policies then apply.

Removing a user from the IdP group only reflects in Vault on the next login/renewal — never instant.

Policy templating parameters (complete list). Inside ACL path strings you may inject:

- `{{identity.entity.id}}`, `{{identity.entity.name}}`
- `{{identity.entity.metadata.<key>}}`
- `{{identity.entity.aliases.<mount_accessor>.id}}`
- `{{identity.entity.aliases.<mount_accessor>.name}}`
- `{{identity.entity.aliases.<mount_accessor>.metadata.<key>}}`
- `{{identity.entity.aliases.<mount_accessor>.custom_metadata.<key>}}`
- `{{identity.groups.ids.<group_id>.name}}`
- `{{identity.groups.names.<group_name>.id}}`
- `{{identity.groups.names.<group_name>.metadata.<key>}}`
- `{{identity.groups.ids.<group_id>.metadata.<key>}}`

Caveats: (1) **wildcards** `(+)` or **globs** `(*)` in the **rendered output** of an identity template yield **"permission denied"** — Vault refuses templated values that resolve to wildcards; (2) **prefer IDs over names** (names change/are reusable); (3) **concurrency race** — mapping a *per-request* claim into shared entity-alias metadata (same alias reused by concurrent jobs) lets one job overwrite another's metadata and break templated policies, so bind such data to distinct entities, not a shared alias.

4. Policies & Authorization

ACL syntax & capabilities. HCL/JSON; deny-by-default. Capabilities: `(create)`, `(read)`, `(update)`, `(patch)`, `(delete)`, `(list)`, `(sudo)`, `(deny)`, and the new `(recover)` (recover data on a path from a snapshot); also `(subscribe)` (+`(subscribe_event_types)`). Precedence:

- `(deny)` **always wins** over any grant.
- Most-specific path match wins**; an exact path beats a glob.

- **Glob** (*) matches only as the **final character**; (+) matches a **single path segment** mid-path (e.g., (secret/+/apikey)).
- Parameter constraints: (required_parameters) (list), (allowed_parameters) (map; (*) key = all), (denied_parameters) (map; takes precedence over allowed). (min_wrapping_ttl)/(max_wrapping_ttl) per path.

Built-in policies.

- **(root)**: superuser, all paths/capabilities; cannot be modified/removed; attached to the initial root token.
- **(default)**: attached to all tokens unless excluded ((token_no_default_policy=true)). Exact contents (from (vault/policy_store.go) (defaultPolicy)):
 - (auth/token/lookup-self) → read; (auth/token/renew-self) → update; (auth/token/revoke-self) → update
 - (sys/capabilities-self) → update
 - (identity/entity/id/{{identity.entity.id}}) → read; (identity/entity/name/{{identity.entity.name}}) → read
 - (sys/internal/ui/resultant-acl) → read
 - (sys/renew) → update; (sys/leases/renew) → update; (sys/leases/lookup) → update
 - (cubbyhole/*) → create, read, update, delete, list
 - (sys/wrapping/wrap) → update; (sys/wrapping/lookup) → update; (sys/wrapping/unwrap) → update
 - (sys/tools/hash) → update; (sys/tools/hash/*) → update
 - (sys/control-group/request) → update
 - (identity/oidc/provider/+/authorize) → read, update

ACL vs Sentinel (CE vs Enterprise). ACL = **Community**. **Sentinel** (logic-based policy-as-code) is **Enterprise-only**: **EGP** (Endpoint Governing Policies — can apply to *unauthenticated* paths and any endpoint) and **RGP** (Role Governing Policies — per token/entity). **Control Groups** (require N approvers before a request proceeds) = **Enterprise-only**. On CE you have ACL + parameter constraints + templating only.

Production least-privilege policies (KV v2 at (secret/); data under (secret/data/...), metadata under (secret/metadata/...)):

(policy-users.hcl) (own KV path via entity-id isolation):

```
hcl
```

```
path "secret/data/users/{{identity.entity.id}}/*" { capabilities = ["create", "read"] }
path "secret/metadata/users/{{identity.entity.id}}/*" { capabilities = ["read", "list"] }
path "secret/metadata/users/{{identity.entity.id}}" { capabilities = ["list"] }
```

policy-staff.hcl (shared team secrets + read app config + own space):

```
hcl

path "secret/data/team/*" { capabilities = ["create", "read", "update", "patch"] }
path "secret/metadata/team/*" { capabilities = ["read", "list", "delete"] }
path "secret/data/appconfig/*" { capabilities = ["read"] }
path "secret/metadata/appconfig/*" { capabilities = ["list"] }
path "secret/data/users/{{identity.entity.id}}/*" { capabilities = ["create", "read", "update"] }
```

policy-admins.hcl (manage auth/policies/identity; DENY root-token creation and destructive sys ops):

```
hcl

path "sys/policies/acl" { capabilities = ["list"] }
path "sys/policies/acl/*" { capabilities = ["create", "read", "update", "delete", "list"] }
path "auth/*" { capabilities = ["create", "read", "update", "delete", "list"] }
path "sys/auth" { capabilities = ["read", "list"] }
path "sys/auth/*" { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "identity/*" { capabilities = ["create", "read", "update", "delete", "list"] }
path "sys/mounts" { capabilities = ["read", "list"] }
path "sys/mounts/*" { capabilities = ["create", "read", "update", "delete", "sudo"] }
path "sys/health" { capabilities = ["read", "sudo"] }
path "sys/audit" { capabilities = ["read", "list", "sudo"] }
path "sys/audit/*" { capabilities = ["create", "read", "update", "delete", "sudo"] }

# Explicit guardrails
path "auth/token/create-orphan" { capabilities = ["deny"] }
path "auth/token/root" { capabilities = ["deny"] }
path "sys/generate-root" { capabilities = ["deny"] }
path "sys/generate-root/*" { capabilities = ["deny"] }
path "sys/step-down" { capabilities = ["deny"] }
path "sys/seal" { capabilities = ["deny"] }
path "sys/storage/raft/snapshot-force" { capabilities = ["deny"] }
```

5. Vault as an OIDC Provider (apps verify Vault-issued JWTs)

Path A — Identity Tokens (simpler; service-to-service / single relying party)

- `identity/oidc/key/<name>`: `rotation_period` (default **24h**), `verification_ttl` (default **24h**), `allowed_client_ids` (* or list), `algorithm` (default **RS256**; also RS384/RS512, ES256/384/512, EdDSA).
- `identity/oidc/role/<name>`: `key`, `template` (base64 Go-template mapping identity → claims), `client_id`, `ttl` (default **24h**).
- Generate: `identity/oidc/token/<role>`. Validate: `identity/oidc/introspect`.
- Discovery: `identity/oidc/.well-known/openid-configuration` + `identity/oidc/.well-known/keys`. Issuer via `identity/oidc/config` (`issuer`).
- Claims populated from entity/group templating → Azure-derived group membership flows into tokens.

Path B — Full OIDC Provider (standards OAuth2 auth-code; best for React+FastAPI)

- `identity/oidc/assignment/<name>`: `entity_ids`, `group_ids`. Built-in `allow_all` permits all entities (immutable). **By default no entities are allowed** until an assignment is attached.
- `identity/oidc/key/<name>`: signs ID tokens (as above).
- `identity/oidc/scope/<name>`: `template` (maps identity metadata → claims). Built-in `openid` scope is immutable.
- `identity/oidc/client/<name>`: `key` (default `default`), `redirect_uris`, `assignments`, `client_type` (default `confidential`; `public` requires PKCE), `id_token_ttl` (default **24h / 86400s**), `access_token_ttl` (default **24h**). Returns generated `client_id` (32 base62 chars) and `client_secret` (`hvo_secret...`, confidential only).
- `identity/oidc/provider/<name>`: `issuer`, `allowed_client_ids`, `scopes_supported`.
- Provider discovery: `.../provider/<name>/.well-known/openid-configuration` + `.../.well-known/keys`. Endpoints: authorization
`<addr>/ui/vault/identity/oidc/provider/<name>/authorize`; token
`<addr>/v1/identity/oidc/provider/<name>/token`; userinfo
`<addr>/v1/identity/oidc/provider/<name>/userinfo`.
`id_token_signing_alg_values_supported`: RS256/384/512, ES256/384/512, EdDSA;
`response_types_supported`: `code`.
- **Behavior**: implements PKCE (required for public, optional for confidential); **first-party clients** → **no consent screen**; authorization code is **single-use**, **~5-minute lifetime**; the issued **access token is a Vault batch token** scoped to read that provider's userinfo; signing keys auto-rotate per the key's `rotation_period`.

How FastAPI/React verify Vault-issued tokens

- **React (SPA, public client + PKCE):** redirect to the authorize endpoint with `code_challenge` → user logs in (already authenticated to Vault via Azure → no second prompt) → receive `code` at `redirect_uri` → exchange at token endpoint with `code_verifier` → get `id_token` (+access token). Store in memory; refresh by re-running the flow (silent if the Vault session is valid).
- **FastAPI (resource server):** read `Authorization: Bearer <id_token>`; fetch and **cache the JWKS** from `.../.well-known/keys`; select key by `kid`; verify **RS256** signature; validate `iss` (provider issuer), `aud` (your client_id), `exp`, `nbf`, `iat`:

python

```
import jwt
from jwt import PyJWKClient
JWKS = "https://vault.example.com/v1/identity/oidc/provider/app/.well-known/keys"
ISSUER = "https://vault.example.com/v1/identity/oidc/provider/app"
AUDIENCE = "<client_id>"
_jwks = PyJWKClient(JWKS) # caches keys
def verify(token: str) -> dict:
    signing_key = _jwks.get_signing_key_from_jwt(token).key
    return jwt.decode(token, signing_key, algorithms=["RS256"],
                      audience=AUDIENCE, issuer=ISSUER,
                      options={"require": ["exp", "iat", "iss", "aud"]})
```

A custom `groups` scope templating `{{identity.entity.groups.names}}` carries the Azure-derived Vault groups, so FastAPI authorizes by group.

When to use which: full provider for browser apps and multi-client OAuth2 with discovery/PKCE/userinfo; identity-token roles for simple machine-to-machine where you just need a signed JWT and control the verifier.

6. Vault as OIDC Client / Identity Broker (consume external IdP)

Use the JWT/OIDC auth method to trust Entra ID/Google/Keycloak/Okta. `role_type=oidc` for interactive browser SSO (Vault does the auth-code dance and gets an access token it can reuse); `role_type=jwt` when a JWT is presented directly (CI, workloads). Pick one verification source per mount: static `jwt_validation_pubkeys`, `jwtks_url` (+`jwtks_ca_pem`), or `oidc_discovery_url`. Bind with `bound_audiences`, `bound_issuer`, `bound_subject`, `bound_claims`; map identity with `user_claim`, `groups_claim`, `claim_mappings`.

7. The Azure AD/Entra ID → Vault → Apps Chain (end-to-end)

Step 1 — Entra app registration. Create an App Registration; record **Application (client) ID** (= `oidc_client_id`) and **Directory (tenant) ID**. Discovery URL:

`https://login.microsoftonline.com/<tenant_guid>/v2.0`. Under **Certificates & secrets** add a client secret (confidential client). Under **Authentication → Redirect URIs (type Web)** add both `<VAULT_ADDR>/ui/vault/auth/oidc/oidc/callback` and `http://localhost:8250/oidc/callback`.

Under **API permissions** add Microsoft Graph **delegated** (`User.Read`) and **Grant admin consent** (the official azuread page prescribes `User.Read` + admin consent for the overage scenario; many deployments also add `GroupMember.Read.All`/`Directory.Read.All`).

Step 2 — Groups claim in Entra. Token configuration → **Add groups claim** → **Security groups** (or All / Groups assigned to the application) → emit into the **ID token**. **Azure emits group OBJECT GUIDs, not names.**

Step 3 — Overage / distributed-claims (full detail). Per Microsoft Learn (ID token claims reference): *"If a user is a member of more groups than the overage limit (150 for SAML tokens, 200 for JWT tokens), then Microsoft Entra ID doesn't emit the groups claim in the token. Instead, it includes an overage claim... that indicates to the application to query the Microsoft Graph API"* (implicit-flow limit is 6 via a `hasgroups` claim). Azure then sends:

json

```
{  "_claim_names": {  "groups": "src1"  },  "_claim_sources": {  "src1": {  "endpoint": "https://graph.windows.net..."  }  } }
```

- **OIDC web login flow (role_type=oidc):** Vault already holds the OAuth access token from the authorization-code exchange, so it **automatically calls Microsoft Graph** at the distributed-claim endpoint to fetch the full group list — **no** `fetch_groups`/`distributed_claim_access_token` **needed**. You need `provider_config={ "provider": "azure" }`, the Graph `User.Read` delegated permission + admin consent, and `oidc_scopes="https://graph.microsoft.com/.default profile"` (the Graph scope makes the access token usable against Graph; `profile` returns the user ID so Vault can build the per-user endpoint URL).
- **JWT login flow (role_type=jwt):** Vault has no access token of its own, so set `provider_config={ "provider": "azure", "fetch_groups": true }` (fetches from Graph `me/getMemberObjects`, also bypassing the per-app 1,000-group assignment limit) **and** pass `distributed_claim_access_token=...` at login.
- **Alternatives to avoid overage:** emit **groups assigned to the application** only; use **app roles** instead of groups (the `roles` claim); or "emit groups as role claims."

Step 4 — Vault OIDC auth config + role (exact commands).

```
bash
```

```
vault auth enable oidc
vault write auth/oidc/config \
  oidc_discovery_url="https://login.microsoftonline.com/<TENANT_GUID>/v2.0" \
  oidc_client_id="<APP_CLIENT_ID>" \
  oidc_client_secret="<CLIENT_SECRET>" \
  default_role="azure" \
  provider_config='{"provider":"azure"}'

vault write auth/oidc/role/azure \
  user_claim="sub" \
  groups_claim="groups" \
  oidc_scopes="https://graph.microsoft.com/.default profile" \
  allowed_redirect_uris="https://vault.example.com/ui/vault/auth/oidc/oidc/callback,h" \
  token_policies="default" token_ttl="1h" token_max_ttl="8h"
```

Step 5 — Three external groups + GUID group aliases + one policy each.

```
bash
```

```
OIDC_ACCESSOR=$(vault auth list -format=json | jq -r '."oidc/".accessor')
for g in users staff admins; do
  GUID=$(vault write -format=json identity/group name="azure-$g" type="external" policy=$OIDC_ACCESSOR | jq -r '.data.guid')
  vault write identity/group-alias name="<AZURE_${g}_GROUP_OBJECT_GUID>" mount_accessor=$OIDC_ACCESSOR
done
```

Step 6 — Claim/alias resolution on login (mechanics). ID token validated (signature via discovery JWKS; iss/aud/exp/nbf with leeways) → `user_claim` (`sub`) becomes the **entity alias name** on the OIDC mount → entity auto-created/looked-up → `groups_claim` values (Azure GUIDs, fetched from Graph if overage) matched against group-aliases on `$OIDC_ACCESSOR` → entity added to matching **external groups** → group **identity policies** applied to the issued token → `claim_mappings` copy chosen claims into entity (alias) metadata.

Step 7 — Chaining to your apps. Once authenticated via Azure, the user's Vault token carries the external-group identity policies. The React app then runs auth-code+PKCE against **Vault's own OIDC provider** (`.../provider/<name>/authorize`); Vault (seeing the user already authenticated) issues an ID token whose `groups` **scope template** (`{{identity.entity.groups.names}}`) surfaces the Azure-derived Vault groups (`azure-users`/`azure-staff`/`azure-admins`); FastAPI verifies and authorizes by group. **Full chain:** Azure AD → (OIDC auth) → Vault entity + external groups + policies → (Vault OIDC provider) → app JWT trusted by FastAPI/React.

8. Token Management & Lifecycle

Service tokens (`(hvs.)`): persisted to storage; **renewable, revocable**; have an **accessor** and **cubbyhole**; can have children; ~96 bytes. Default for humans/admin. **Batch tokens** (`(hvb.)`): **encrypted blob, not stored; not renewable, not listable, no accessor, no cubbyhole, no children**; lightweight (~128 bytes); **recommended for high-volume machine auth**. Orphan batch tokens replicate to perf/DR clusters; non-orphan batch tokens are bound to their parent's cluster and expire with the parent. **Recovery tokens** (`(hvr.)`): only in recovery mode; direct storage access for break-glass.

	Service	Batch
Stored	Yes	No
Renewable	Yes	No
Accessor/cubbyhole	Yes	No
Child tokens	Yes	No
Overhead	Heavier	Minimal
Use	Humans, long-lived	Machines, ephemeral, high-scale

Hierarchy & revocation: tokens form parent→child trees; revoking a parent **cascades** to children and their leases. **Orphan tokens** (`(-orphan)`, requires sudo/root) have no parent and survive parent revocation.

TTL mechanics: precedence `(token_explicit_max_ttl) > mount (max_lease_ttl) > system (max_lease_ttl)` (default 32 days = 768h). Periodic tokens (`(-period)`/`(token_period)`): **no max TTL**, TTL resets to the period on each renewal — never expire while renewed (use for long-running services/agents, **never humans**). Renewal extends by `(increment)` (capped by max).

Token roles (`(auth/token/roles/<n>)`): `(allowed_policies)`, `(disallowed_policies)`, `(orphan)`, `(renewable)`, `(path_suffix)`, `(allowed_entity_aliases)`, `(token_period)`, `(token_explicit_max_ttl)`, `(token_no_default_policy)`, `(token_num_uses)`, `(token_type)`.

Accessors: lookup/renew/revoke a token *without* its value; **safe to log**; audit devices can be told not to HMAC accessors for correlation. Listing tokens is only via `(auth/token/accessors)`.

Revocation: `(vault token revoke <token>)`; `(vault token revoke -accessor <acc>)`; `(vault lease revoke -prefix <path>)`; `(vault token revoke -mode=orphan)`.

Best practices: short TTLs + auto-renewal via Vault Agent; revoke tokens on app shutdown; never give humans periodic tokens; write tokens to a **sink file**, not env vars; revoke the **root token** after

setup.

9. Vault Agent & Secret Delivery

Auto-auth: `(auto_auth { method "approle" { config = { role_id_file_path, secret_id_file_path, secret_id_response_wrapping_path, remove_secret_id_file_after_reading (default true) } } sink "file" { config = { path, mode } } })`. The Agent authenticates, writes the token to the sink, and keeps it renewed/re-authenticated. `(wrap_ttl)` on the **method** protects the creation_path (no renew by agent) vs on the **sink** (agent can renew; cleartext between Vault and agent). `(cache { use_auto_auth_token = true })` lets local apps hit the agent's API proxy without handling tokens. **Templating:** Consul-Template syntax renders secrets to files (`(template { contents/source, destination })`). **Comparison:** Vault Agent (sidecar/host daemon, any platform) vs **Vault Secrets Operator** (K8s CRDs sync to K8s Secrets) vs **Agent Injector** (K8s mutating webhook injects an agent sidecar). **FastAPI consumption pattern (recommended):** run Vault Agent alongside the app; auto-auth via AppRole (wrapped SecretID); either render secrets to **tmpfs** the app reads at startup, or expose the cached token via the local API proxy for short-lived/dynamic secrets. Prefer files/in-memory over env vars (env leaks via `(/proc)`, crash dumps, child processes). Fetch at runtime; never commit.

10. Leases & Dynamic Secrets Overview

Dynamic secrets and some auth return a **lease** (`(lease_id)`, `(lease_duration)`, `(renewable)`). `(sys/leases/lookup)`, `(.../renew)`, `(.../revoke)`, `(.../revoke-prefix)`, `(.../revoke-force)`. Tune `(default_lease_ttl)`/`(max_lease_ttl)` at the system (config) and per-mount level (`(vault secrets tune -max-lease-ttl=)`). Revoking a token revokes its leases. **Lease-count quotas** (`(sys/quotas/lease-count)`) cap active leases and **are available in Community**. Even for an authn/authz-only use, dynamic engines (database, PKI, transit) share this lease machinery — e.g., PKI for issuing TLS certs, transit for auto-unseal/encryption-as-a-service.

11. Production Storage & High Availability

File backend: single-node, **no HA, snapshots unsupported** — lab only; migrate before production.

Integrated Storage (Raft) — recommended. `(storage "raft" { path, node_id, retry_join { leader_api_addr, leader_ca_cert, leader_client_cert_file, leader_client_key_file, auto_join, auto_join_scheme, auto_join_port }, performance_multiplier, max_entry_size (default 1MB), autopilot_reconcile_interval, snapshot_threshold (8192), trailing_logs (10000) })`. **Requires** `(cluster_addr)` (e.g. `(https://<node>:8201)`); a separate `(ha_storage)` cannot be

declared. Set `(disable_mlock = true)` and **disable swap** (mlock interacts badly with BoltDB memory-mapped files).

- Operator commands: `(vault operator raft list-peers)`, `(join)`, `(remove-peer <id>)` (must wipe that node's data before rejoining), `(autopilot get-config/set-config/state)`.
- **Autopilot defaults:** `(cleanup_dead_servers)` (false), `(dead_server_last_contact_threshold)` (24h — keep high; active only when cleanup enabled), `(last_contact_threshold)` (10s), `(max_trailing_logs)` (1000), `(min_quorum)` (no default; set to expected voter count when enabling cleanup), `(server_stabilization_time)` (10s), `(disable_upgrade_migration)` (Enterprise-only field).
- **Quorum:** odd node counts — **3 nodes survive 1 failure, 5 survive 2**. Even counts add no fault tolerance and risk split decisions. Quorum loss = no writes.

`(operator migrate)` (file → Raft): offline; `(migrate.hcl)` with `(storage_source)` and `(storage_destination)`; stop Vault, run `(vault operator migrate -config=migrate.hcl)`, set the new `(storage "raft" + (cluster_addr))`, restart.

Snapshots: `(vault operator raft snapshot save <file>)` / `(restore [-force] <file>)` is the **only supported Raft backup** and is available in **all editions**. **Automated/scheduled snapshots are Enterprise-only** — on CE script `(snapshot save)` via cron, ship offsite, and test restores.

HA architecture: active/standby; standbys forward/redirect to the active node; inter-node traffic on **cluster port 8201**. Put a load balancer in front using `(sys/health)`, but **do NOT terminate TLS at the LB** — TCP passthrough so encryption is end-to-end to Vault.

12. Auto-Unseal

Transit auto-unseal — `(seal "transit" { address (req), token (req; via VAULT_TOKEN), key_name (req), mount_path (req, e.g. "transit/"), namespace (""), disable_renewal ("false"), tls_ca_cert, tls_client_cert, tls_client_key, tls_server_name, tls_skip_verify ("false") })`. On the "unseal Vault": `(vault secrets enable transit; vault write -f transit/keys/autounseal)`; a policy granting **(update)** on **(transit/encrypt/autounseal)** and **(transit/decrypt/autounseal)**; create an **orphan, periodic** token (`(vault token create -orphan -policy=autounseal -period=24h)`) so it never hits a max TTL and won't break if a parent is revoked.

Cloud KMS seals:

- `(seal "awskms" { region ("us-east-1"), kms_key_id (req; ARN or alias/), endpoint, access_key/secret_key/session_token or instance role, disabled })`
- `(seal "azurekeyvault" { tenant_id (req), client_id (req or MSI), client_secret (req or MSI), environment ("AZUREPUBLICCLOUD"), vault_name (req), key_name (req), resource })`
— the app needs **Key Vault Crypto User + Key Vault Secrets User** roles.

- `seal "gcpckms" { project, region, key_ring, crypto_key, credentials }; seal "ocikms" { ... }; seal "transit" { ... }`

Recovery keys vs unseal keys. With auto-unseal, init produces **recovery keys** (not unseal keys). They authorize **generate-root, rekey, reseal** but **cannot decrypt the root key** (the KMS does). Manual Shamir unseal uses **unseal keys** that *do* reconstruct the root key.

Seal migration: to move Shamir↔auto-unseal or auto↔auto, mark the old seal `disabled = true`, add the new seal stanza, restart with `vault operator unseal -migrate` supplying threshold keys; Vault re-encrypts the root key under the new seal. Followers don't need `-migrate`.

Catastrophic risk: deleting/disabling the KMS key makes the cluster unrecoverable even from backups. Guard the key with strict IAM/SCPs, deletion protection, and key-rotation that retains old key versions (transit: never disable/delete old key versions).

CE vs Enterprise: CE supports a **single** auto-unseal seal; **multi-seal HA (2-3 seals) and seal-wrapping** are Enterprise-only.

13. Audit & Observability

Audit devices (`(vault audit enable <type>)`):

- **file:** `file_path` (req), `log_raw` (default **false** — keep false), `hmac_accessor` (default **true**), `mode` (e.g. 0600), `format` (`json` | `jsonx`), `prefix`, `elide_list_responses` (recommend **true**).
- **syslog:** `facility` ("AUTH"), `tag` ("vault"), `log_raw`, `hmac_accessor`, `format`. Use a **TCP** listener (UDP truncates large entries).
- **socket:** `address`, `socket_type` (`tcp` | `udp` | `unix`), `log_raw`, `hmac_accessor`, `format`. Prefer a **local unix socket** to avoid loss.

CRITICAL behavior: Vault guarantees writing each entry to ≥1 enabled device; **if ALL audit devices fail to write, Vault BLOCKS the request** (security over availability). Therefore run **at least TWO devices on independent storage/types**, and deduplicate across them by `.request.id`.

HMAC: most string values are HMAC-SHA256-salted by default; **non-string values (ints/bools) are NOT hashed** — send sensitive numbers as quoted strings. Use `sys/audit-hash/<device>` to compute the HMAC of a known value to match in logs.

`audit_non_hmac_request_keys`/`audit_non_hmac_response_keys` (mount tuning) exempt chosen fields. Some endpoints are never audited (sys/init, seal-status, unseal, health, rekey-init, etc.).

Log management: `chmod 0600`, dedicated volume/partition, **logrotate + SIGHUP**, ship to SIEM (Splunk/Elastic/Datadog), checksum/immutability. Audit logs ≠ **operational/server logs** (process/config events).

Telemetry (`(telemetry { ... })`): `(prometheus_retention_time)` (e.g. "24h"), `(disable_hostname)` (true), `(usage_gauge_period)`, `(maximum_gauge_cardinality)`, `(statsd_address)`, `(dogstatsd_addr)`/`(dogstatsd_tags)`, `(stackdriver_*)`. Expose `/v1/sys/metrics?format=prometheus`; the listener's `(telemetry { unauthenticated_metrics_access = true })` can expose metrics without a token (keep it network-restricted). Watch audit-failure metrics, token/lease counts, seal status, Raft leadership/commit times.

14. End-to-End TLS

listener "tcp" TLS fields (defaults): `(address)` ("127.0.0.1:8200"), `(cluster_address)` (:8201), `(tls_disable)` (**false**), `(tls_cert_file)` (leaf+intermediate chain, **leaf first**), `(tls_key_file)`, `(tls_min_version)` ("**tls12**"), `(tls_max_version)` ("**tls13**"), `(tls_cipher_suites)` (" — **ignored under TLS 1.3**; for 1.2 exclude 3DES to avoid Sweet32/CVE-2016-2183), `(tls_require_and_verify_client_cert)` ("**false**"), `(tls_client_ca_file)` (""), `(tls_disable_client_certs)` ("false" — **mutually exclusive** with `require_and_verify`), `(tls_prefer_server_cipher_suites)` (**deprecated, no effect**), `(x_forwarded_for_*)` (trusted proxies). Certs **reload on SIGHUP**. To use a non-default 1.2 cipher set, pin `(tls_min_version=tls_max_version="tls12")` and list ciphers (e.g. the ECDHE-AES-GCM/CHACHA20 set), since 1.3 doesn't accept explicit ciphers.

mTLS: for cert-auth clients set `(tls_require_and_verify_client_cert=true)` + `(tls_client_ca_file)`. Use an internal CA (or Vault PKI) to issue the server cert; the **SAN must match the hostname clients use** and your `(api_addr)`/issuer hostname.

LB guidance: do NOT terminate TLS at the LB — TCP passthrough so traffic is encrypted end-to-end to Vault. Set HSTS via `(listener { custom_response_headers { "default" = { "Strict-Transport-Security" = ["max-age=31536000; includeSubDomains"] } } })`.

Docker: mount cert/key **read-only**, set `(VAULT_CACERT)` and `(VAULT_ADDR=https://...)`.

15. Docker / Compose Hardening (your exact setup)

Your compose (`(hashicorp/vault:1.21)`, `(cap_add: IPC_LOCK)`, `(user: 100:100)`, `(SKIP_SETCAP=true)`, `(command: server)`, config+data bind mounts, custom network). Issues & fixes:

- **The bare `(server)` command needs a config**: supply `(command: server - config=/vault/config/vault.hcl)` (the entrypoint reads HCL/JSON from `(/vault/config)`; or set `(VAULT_LOCAL_CONFIG)`, which is written to `(/vault/config/local.json)`). Without a config in server mode it won't start properly.
- **1.21 mlock/IPC_LOCK change**: the official image **removed** `(cap_ipc_lock)` at build time so Vault in containers **cannot call** `(mlock())`; HashiCorp guidance: set `(disable_mlock = true)`

and **disable or encrypt swap** on the host. Combined with the separate change that `disable_mlock` now has no default and is required with **Integrated Storage**, you **must** set it explicitly or Vault fails to start. Therefore set `disable_mlock = true` in the HCL and you can **drop the `IPC_LOCK` capability** and the `SKIP_SETCAP` workaround. The image runs as non-root **UID 100** by default. (Note: `cap_ipc_lock` was toggled across point releases — 1.21.6 set it, 1.21.7 removed it — so pin and test your exact tag.)

Complete production server config (`vault.hcl`):

```
hcl
```

```

ui            = true
disable_mlock = true                # REQUIRED with raft in 1.21; swap must be off/e
cluster_name  = "prod-vault"
log_level     = "info"
log_format    = "json"

storage "raft" {
    path      = "/vault/data"
    node_id   = "vault-1"
    # retry_join { leader_api_addr = "https://vault-2:8200" } # multi-node
}

listener "tcp" {
    address            = "0.0.0.0:8200"
    cluster_address    = "0.0.0.0:8201"
    tls_disable        = false
    tls_cert_file      = "/vault/tls/vault.crt"    # leaf+chain, leaf first
    tls_key_file       = "/vault/tls/vault.key"
    tls_min_version    = "tls12"
    tls_disable_client_certs = true
    custom_response_headers {
        "default" = { "Strict-Transport-Security" = ["max-age=31536000; includeSubDomains"]
    }
}

seal "transit" {                    # or awskms/azurekeyvault/gcpckms
    address          = "https://transit-vault:8200"
    key_name         = "autounseal"
    mount_path       = "transit/"
    # token via VAULT_TOKEN env (orphan+periodic)
}

api_addr      = "https://vault.example.com:8200"
cluster_addr  = "https://vault-1:8201"
max_lease_ttl    = "768h"
default_lease_ttl = "168h"
plugin_directory = "/vault/plugins"
telemetry { prometheus_retention_time = "24h" disable_hostname = true }
# entropy "seal" { mode = "augmentation" } # Enterprise-only – do NOT use on CE

```

Compose hardening checklist: set `disable_mlock=true` and **drop** `IPC_LOCK`; run as **non-root** UID **100**; mount config + TLS `:ro`; persist raft `(/vault/data)` and audit logs `(/vault/logs)` on

volumes; expose **only 8200** (and 8201 if clustering) on a **private** network; set

`(VAULT_ADDR)/(VAULT_CACERT)`; **never pass root/unseal/recovery keys via env**; `(read_only: true)` root filesystem + `(tmpfs)` scratch; **disable host swap** (or encrypt); `(restart: unless-stopped)`; healthcheck on `(/v1/sys/health)`; CPU/memory `(limits)`; **no** `(docker exec)`/SSH (hardened jump box); **pin the image by digest** `(hashicorp/vault:1.21@sha256:...)`.

Reference HashiCorp's **Production Hardening** guide point by point: don't run as root; minimal write privileges; end-to-end TLS + HSTS; disable swap; disable core dumps `(LimitCORE=0)`; single tenancy; firewall to NTP + Vault only; ≥ 1 (ideally 2) audit devices; upgrade frequently + subscribe to security announcements; synchronized clocks (NTP); SELinux/AppArmor; tuned ulimits.

16. Security Hardening (consolidated checklist — #1 priority)

- **Don't run as root** — the 1.21 image is already UID 100; keep it.
- **Root token**: use only for bootstrap, then `(vault token revoke <root>)`. Regenerate on demand with `(operator generate-root)` + recovery/unseal keys. The 1.21 change: `(sys/generate-root)`, `(sys/rekey)`, and **DR-operation-token endpoints are now authenticated by default**; `(enable_unauthenticated_access)` can revert — **don't**. (DoS bulletin HCSEC-2026-08, affecting Vault CE up to 1.21.4 and fixed in 2.0.0, motivated tightening these.)
- **Least privilege everywhere**: minimal policies, separate user/app policies, limit wildcards, **entity-id templated isolation** for per-user paths.
- **Secret zero / response wrapping**: wrapped SecretIDs, short TTL, `(secret_id_num_uses=1)`, CIDR-bound; deliver RoleID and SecretID over **separate channels**; remove the wrapping token after use; **alert on unwrap failures / response-wrapping mismatches** (tamper evidence) and on auth errors.
- **End-to-end TLS, HSTS** via `(custom_response_headers)`, TLS 1.3 where clients allow.
- **Disable swap and core dumps** (sensitive data in memory; doubly important with Raft + no mlock in containers).
- **≥ 2 audit devices on independent storage**, `(log_raw=false)`, ship to SIEM, monitor audit-failure metrics.
- **Restrict storage access**, single tenancy, no direct `(docker exec)`/SSH, hardened jump box.
- **NTP clock sync** (TTLs, cert validity, JWT exp/nbf). **Upgrade frequently**; subscribe to the HashiCorp security-announcement list; watch the CHANGELOG.
- **Off-boarding**: removing an IdP account does **not** instantly revoke live Vault tokens. Remove the user from IdP groups (reflected next login/renewal), **revoke their leases/tokens** (by accessor or entity), **delete the canonical entity**, and disable the auth mount to revoke *all* its tokens if needed.
- **TTL/lease management**: smallest workable default/max lease TTLs, tune at mount + role, **lease-count quotas** `(sys/quotas/lease-count)`, CE).

- **MFA & rate limiting:** basic TOTP login MFA is available in Community; step-up / login-enforcement MFA is Enterprise. Rate-limit quotas AND lease-count quotas (`sys/quotas`) are both available in Community. User/password lockout is on by default for approle/ldap/userpass (tune threshold/duration).
-

17. Practical End-to-End Setup Walkthrough (for this user)

Target architecture: Integrated Storage (Raft) single node (grow to 3) + **transit or cloud-KMS auto-unseal** + end-to-end TLS + **two audit devices** + Azure OIDC for 3 human roles → 3 external groups → 3 least-privilege policies + **AppRole (batch tokens, wrapped CIDR-bound SecretIDs)** for machines + **Vault OIDC provider** issuing JWTs to FastAPI/React + **root token revoked**.

Bootstrap sequence (post init/unseal, as root token):

```
bash
```

```
export VAULT_ADDR=https://vault.example.com:8200 VAULT_CACERT=/path/ca.pem
vault login <root>

# 1. Audit (two devices) FIRST
vault audit enable file file_path=/vault/logs/audit.log
vault audit enable -path=syslog syslog tag="vault" facility="AUTH"

# 2. KV v2
vault secrets enable -path=secret -version=2 kv

# 3. Policies
vault policy write policy-users policy-users.hcl
vault policy write policy-staff policy-staff.hcl
vault policy write policy-admins policy-admins.hcl

# 4. Azure OIDC auth + role (Section 7, Step 4)

# 5. Three external groups + GUID aliases (Section 7, Step 5)

# 6. AppRole for machine services
vault auth enable approle
vault write auth/approle/role/fastapi-svc token_policies="policy-app" \
  token_type="batch" token_ttl="20m" token_max_ttl="20m" \
  secret_id_ttl="10m" secret_id_num_uses=1 \
  secret_id_bound_cidrs="10.0.0.0/24" token_bound_cidrs="10.0.0.0/24"
vault read auth/approle/role/fastapi-svc/role-id
vault write -wrap-ttl=120s -f auth/approle/role/fastapi-svc/secret-id # deliver wrap

# 7. Vault OIDC provider for the apps
vault write identity/oidc/key/app-key rotation_period="24h" verification_ttl="24h" allowed_client_ids="*"
vault write identity/oidc/assignment/app-assign group_ids="$(vault read -field=id identity/oidc/group/external)"
vault write identity/oidc/scope/groups template="$(echo '{"groups":{{identity.entity.aliases}}}' | jq -r)"
vault write identity/oidc/client/webapp key="app-key" assignments="app-assign" \
  redirect_uris="https://app.example.com/callback,http://localhost:5173/callback" \
  client_type="public" id_token_ttl="1h" access_token_ttl="1h"
CLIENT_ID=$(vault read -field=client_id identity/oidc/client/webapp)
vault write identity/oidc/provider/app issuer="https://vault.example.com" \
  allowed_client_ids="$CLIENT_ID" scopes_supported="groups"

# 8. Revoke the root token
vault token revoke -self
```

Operational runbook (ordered): (1) provision host (swap off/encrypted, NTP, firewall); (2) deploy compose with `(disable_mlock=true)`, TLS, auto-unseal; (3) `(operator init)` (store recovery keys split among ≥ 3 custodians, offline); (4) confirm auto-unseal on restart; (5) enable **two** audit devices; (6) run bootstrap; (7) **revoke root**; (8) cron `(operator raft snapshot save)` offsite + test restores; (9) monitor telemetry + audit-failure alerts; (10) upgrade promptly on security bulletins.

Hardening checklist (final): UID 100 non-root ✓; root revoked ✓; TLS end-to-end, no LB termination ✓; `(disable_mlock=true)` + swap off ✓; drop IPC_LOCK ✓; image pinned by digest ✓; config/TLS mounted `(:ro)` ✓; two audit devices, `(log_raw=false)`, SIEM ✓; least-privilege templated policies ✓; AppRole batch tokens + wrapped/CIDR/use-limited SecretIDs ✓; KMS key protected by IAM + deletion protection ✓; lease-count + rate-limit quotas ✓; user-lockout + TOTP MFA ✓; NTP ✓; generate-root/rekey left authenticated ✓.

Recommendations

1. **Now (week 1):** Stand up single-node Raft + **transit auto-unseal** (a tiny second Vault as the transit "unseal Vault", or a cloud KMS) + end-to-end TLS + **two audit devices**. Fix the compose: add `(-config=/vault/config/vault.hcl)`, set `(disable_mlock=true)`, drop `(IPC_LOCK)`, mount TLS/config `(:ro)`, pin the image digest, disable host swap. **Revoke the root token** after bootstrap.
 2. **Authn/authz (week 2):** Wire Azure OIDC → 3 external groups via **GUID** group-aliases → 3 least-privilege policies. Validate the **>200-group overage** path works (automatic in the OIDC web flow with `(User.Read)` + `(oidc_scopes="https://graph.microsoft.com/.default profile")`). Stand up the **Vault OIDC provider** and verify FastAPI JWKS verification + React PKCE flow.
 3. **Machines (week 3):** AppRole per service, **batch tokens**, wrapped/CIDR-bound/single-use SecretIDs delivered via **Vault Agent auto-auth**; render secrets to tmpfs, not env.
 4. **Resilience:** cron `(raft snapshot save)` offsite (automated snapshots are Enterprise — script it on CE) and **test restores**. Grow to **3 (or 5) Raft nodes**; set autopilot `(cleanup_dead_servers=true)` + `(min_quorum=3)`.
 5. **Thresholds that change the plan:** if you need Sentinel/EGP-RGP, control groups, namespaces, replication, automated snapshots, multi-seal HA, HSM, or step-up MFA → that's **Vault Enterprise** (or evaluate **OpenBao**, the MPL-2.0 fork, which includes namespaces free). For free self-hosted authn/authz only, **CE is sufficient**.
-

Caveats

- **Licensing/lineage:** HashiCorp moved Vault from MPL 2.0 to **BSL 1.1 on August 10, 2023** (announced via GlobeNewswire; Vault 1.14 was the last MPL release, 1.15+ ships under BUSL). BSL is source-available, **not** OSI open-source (it prohibits competing/hosted

offerings). **IBM completed its \$6.4 billion all-cash (\$35/share) acquisition of HashiCorp on February 27, 2025** (IBM newsroom; UK CMA cleared it Feb 25, 2025, FTC already cleared) — recent docs/CHANGELOGs reference IBM (e.g., Passport Advantage license keys). The release line after 1.21 moves to **2.0 / IBM lifecycle versioning** (e.g., the HCSEC-2026-08 DoS fix is "fixed in 2.0.0"). **OpenBao**, the MPL-2.0 fork forked from Vault 1.14.0, is now governed under the Linux Foundation (OpenSSF Sandbox project as of June 2025), reached **v2.5.0 on February 4, 2026**, is API-compatible, and includes namespaces and horizontal read scalability free — a credible CE alternative.

- **Version-specific behaviors to verify against your exact patch release:** the `disable_mlock`-required + container `cap_ipc_lock`-removed changes (the cap was toggled across 1.21.6/1.21.7); authenticated-by-default generate-root/rekey/DR-token; non-canonical path rejection/redirect; the **LIST-with-trailing-slash deny fix** (audit your deny policies); the new `recover` capability. Pin and test your specific image tag.
- **Azure overage specifics:** the official azuread page prescribes `User.Read` **delegated** + **admin consent** for >200 groups; some community guides add `GroupMember.Read.All`/`Directory.Read.All`. The doc shows two different `oidc_scopes` values in different sections — use the **Graph scope** (`https://graph.microsoft.com/.default` `profile`) for the Graph-resolution path. Microsoft's documented limits are **200 for JWT**, **150 for SAML**, and **6 for implicit flow** (`hasgroups`). Test with your tenant.
- **Forward-looking items flagged, not asserted as live:** AI-agent/OAuth-resource-server features and SCIM appear as **Beta/Enterprise** in 2.0.0 release notes; PKI-only licensing mode is Enterprise 1.21. Treat as not-applicable to your CE authn/authz build.
- **Operational gotchas:** the **identity-templating race** (concurrent reuse of one alias's metadata) and the **eventual (not instant) external-group sync** are real — design around them (distinct entities for concurrent machine identities; rely on token revocation, not IdP removal, for instant cutoff).